AI Response

Provider: openai
Model: gpt-5
Original: START_testnewmodel_openai.docx
Generated: 2026-09-03T03:28:59.069Z

---

a) Intended function and scheduling

- Function: The sensor thread blocks on getchar(), and whenever it reads 'e' or 'n' it writes the character into the shared structure and increments a counter. The main thread executes one step of a simple traffic-light state machine per loop. In state EWG_NSR it waits for a North–South car by checking shared.key for 'n', then advances through yellow/red/green/yellow with sleeps.
- Intended scheduling for the sensor thread: round-robin policy at priority 20, as indicated by the attribute setup and the in-code comment.

b) Three distinct defects and their single-CPU runtime consequences

1. Scheduling attributes are ignored

- Defect: The code never sets the scheduling inheritance to explicit. Without pthread_attr_setinheritsched(..., PTHREAD_EXPLICIT_SCHED), the created thread inherits the creator’s scheduling, and the SCHED_RR+priority 20 settings are not applied.
- Consequence: The sensor thread won’t run at the intended round-robin, priority-20 level. On a single CPU, it may run at the main thread’s lower, default priority and/or policy, reducing sensor responsiveness and increasing latency for handling input.

2. Unsynchronized access to shared data (data race)

- Defect: Both threads read/write shared.key and shared.count without any synchronization.
- Consequence: Concurrent reads/writes can interleave unpredictably. The main thread can read stale values or the sensor thread’s write to key can race with the main thread’s reset to 0, causing missed detections and inconsistent counting. This can manifest as the state machine never leaving EWG_NSR or advancing at the wrong times.

3. Busy-wait (spin) on shared.key in the main thread

- Defect: The EWG_NSR case spins in while (shared.key != 'n') with no blocking or yielding.
- Consequence: On a single CPU, a spinning main thread at the same or higher priority can monopolize the processor. If the policy is FIFO, it can starve the sensor thread until the main thread blocks elsewhere; even under round robin, it wastes CPU and increases input-to-action latency. It also exacerbates the data race on key by continuously reading without synchronization.

c) Code corrections (snippets only)

1. Make the scheduling attributes take effect

- Replace the thread attribute setup with explicit inheritance:

pthread_attr_init(&sensor_attr); pthread_attr_setschedpolicy(&sensor_attr, SCHED_RR); sensor_param.sched_priority = 20; pthread_attr_setschedparam(&sensor_attr, &sensor_param); pthread_attr_setinheritsched(&sensor_attr, PTHREAD_EXPLICIT_SCHED); /* apply RR, prio 20 */

pthread_create(&sensor_id, &sensor_attr, sensor_thread, NULL);

2. Protect the shared structure and avoid busy-wait by using a mutex + condition variable

- Declarations (global):

static pthread_mutex_t shared_mtx = PTHREAD_MUTEX_INITIALIZER; static pthread_cond_t shared_cv = PTHREAD_COND_INITIALIZER;

- Sensor thread (write under lock and signal):

void *sensor_thread(void *arg) { int c; while (1) { c = getchar(); if (c == 'e' || c == 'n') { pthread_mutex_lock(&shared_mtx); shared.key = (char)c; shared.count = shared.count + 1; pthread_cond_signal(&shared_cv); /* notify waiter */ pthread_mutex_unlock(&shared_mtx); } } return NULL; }

- State machine (EWG_NSR): wait without spinning, then consume the event atomically:

case EWG_NSR: printf("East-West GREEN , North-South RED\n"); pthread_mutex_lock(&shared_mtx); while (shared.key != 'n') { /* release CPU while waiting */ pthread_cond_wait(&shared_cv, &shared_mtx); } shared.key = 0; /* consume */ pthread_mutex_unlock(&shared_mtx); *CurState = EWY_NSR; break;

d) Priority inversion and the preferred primitive

- Priority inversion: A higher-priority thread is blocked waiting for a resource held by a lower-priority thread, while one or more medium-priority threads run, preventing the low-priority holder from ever running to release the resource. The effective result is that the high-priority thread is indirectly delayed by lower-priority activity.
- Choice: Use a QNX mutex rather than an unnamed semaphore. In a priority-driven real-time system, a mutex can provide priority inheritance so the low-priority holder temporarily executes at a higher priority to release the lock, mitigating inversion. Semaphores do not adjust a thread's effective priority and thus don't address inversion; they also incur higher overhead than mutexes for simple mutual exclusion.

References

lab-2/LAB-2-REQUIREMENT.md QNX Momentics IDE 7.0 User's Guide/System-Architecture/Thread-Scheduling/Thread scheduling.md QNX Momentics IDE 7.0 User's Guide/System-Architecture/Thread-Scheduling/Scheduling policies.md QNX Momentics IDE 7.0 User's Guide/System-Architecture/Synchronization-services/Synchronization services.md QNX Momentics IDE 7.0 User's Guide/System-Architecture/Synchronization-services/Semaphores.md QNX Momentics IDE 7.0 User's Guide/System-Architecture/Synchronization-services/Synchronization via scheduling policy.md